

EEBus Technical Specification

SHIP Pairing Service

Version 1.0.0

Cologne, 2025-12-03

EEBus Initiative e.V.

Deutz-Mülheimer-Straße 183
51063 Cologne
GERMANY

Rue d'Arlon 25
1050 Brussels
BELGIUM

Phone: +49 221 / 715 938 - 0
Fax: +49 221 / 715 938 - 99

info@eebus.org
www.eebus.org

District court: Cologne
VR 17275

Terms of use for publications of EEBus Initiative e.V.**General information**

The specifications, particulars, documents, publications and other information provided by the EEBus Initiative e.V. are solely for general informational purposes. Particularly specifications that have not been submitted to national or international standardisation organisations by EEBus Initiative e.V. (such as DKE/DIN-VDE or IEC/CENELEC/ETSI) are versions that have not yet undergone complete testing and can therefore only be considered as preliminary information. Even versions that have already been published via standardisation organisations can contain errors and will undergo further improvements and updates in future.

Liability

EEBus Initiative e.V. does not assume liability or provide a guarantee for the accuracy, completeness or up-to-date status of any specifications, data, documents, publications or other information provided and particularly for the functionality of any developments based on the above.

Copyright, rights of use and exploitation

The specifications provided are protected by copyright. Parts of the specifications have been submitted to national or international standardisation organisations by EEBus Initiative e.V., such as DKE/DIN-VDE or IEC/CENELEC/ETSI, etc. Furthermore, all rights to use and/or exploit the specifications belong to the EEBus Initiative e.V., Deutz-Mülheimer-Straße 183, 51063 Cologne, Germany and can be used in accordance with the following regulations:

The use of the specifications for informational purposes is allowed. It is therefore permitted to use information evident from the contents of the specifications. In particular, the user is permitted to offer products, developments and/or services based on the specifications.

Any respective use relating to standardisation measures by the user or third parties is prohibited. In fact, the specifications may only be used by EEBus Initiative e.V. for standardisation purposes. The same applies to their use within the framework of alliances and/or cooperations that pursue the aim of determining uniform standards.

Any use not in accordance with the purpose intended by EEBus Initiative e.V. is also prohibited.

Furthermore, it is prohibited to edit, change or falsify the content of the specifications. The dissemination of the specifications in a changed, revised or falsified form is also prohibited. The same applies to the publication of extracts if they distort the literal meaning of the specifications as a whole.

It is prohibited to pass on the specifications to third parties without reference to these rights of use and exploitation.

It is also prohibited to pass on the specifications to third parties without informing them of the authorship or source.

Without the prior consent of EEBus Initiative e.V., all forms of use and exploitation not explicitly stated above are prohibited.

1	Table of contents	
2	Table of contents.....	3
3	List of figures	4
4	List of tables	4
5	1 Introduction.....	5
6	2 References.....	6
7	2.1 EEBUS documents	6
8	2.2 Normative References	6
9	3 Terms and Definitions	7
10	4 Process description	8
11	4.1 Roles and responsibilities	8
12	4.2 Process steps.....	9
13	4.3 addCu-request process deactivation/reactivation at devA	11
14	4.4 Parallel shippairing requests of devZ.....	12
15	4.5 Handling absence of SHIP Message Exchange at devZ	12
16	5 shippairing service.....	13
17	5.1 Introduction	13
18	5.2 Service instance names.....	13
19	5.3 General service requirements.....	14
20	5.4 TXT record	15
21	5.5 Correcting an unsuccessful request.....	17
22	6 Identification and cryptographic requirements	19
23	6.1 Identification	19
24	6.2 Cryptographic parameters	19
25	6.3 Random number generators.....	20
26	7 shippairing digest calculation.....	22
27	7.1 Overview	22
28	7.2 Select algorithm	22
29	7.3 Create (temporary) key K.....	22
30	7.4 Create (temporary) message M.....	23
31	7.5 Calculate the digest.....	23
32	8 Create shippairing request	24
33	9 Evaluate shippairing request.....	25
34	10 Update SHIP trust.....	27
35	10.1 Introduction	27
36	10.2 Verification of fingerprint during TLS handshake	27
37	10.3 Remove SHIP trust of past devZ.....	27
38	10.4 Single trust store for SHIP and SHIP Pairing Service (informative)	27
39	11 Ring buffer of devA.....	29
40	11.1 Introduction	29
41	11.2 Definition of ring buffer and "next"	29
42	11.3 Adding a new entry to the ring buffer	29
43	11.4 Get entry of currently trusted devZ.....	30
44	12 SHIP QR code extensions.....	31
45	Annex A – Example values.....	32
46	A.1 Example values for devA.....	32

47	A.2	Example values for devZ	33
48	A.3	shippairing request preparations of devZ for devA	34
49		Annex B – Pairing without QR code (informative)	35
50		Annex C – Security aspects	36
51	C.1	Security of SHIP SKIs (informative)	36
52	C.2	Certificate fingerprint	36
53	C.3	shippairing as migration step from SKI to fingerprint.....	37
54			

55 List of figures

56	Figure 1: Overview of shippairing process (without SHIP details)	8
57	Figure 2: QR code example for devA	33
58		
59		

60 List of tables

61	Table 1: Key-value pairs of a shippairing TXT Record.....	15
62	Table 2: SHIP QR code extensions for fingerprints.....	31
63		

64

1 Introduction

This document describes a mechanism to trust a SHIP node ([SHIP]) based upon a shared secret.

The conventional procedure to establish trust between two SHIP nodes devA and devZ requires that the nodes know the respective other SHIP ID and trust the respective other node's certificate or the related SKI or public key. Usually, the user of these nodes can start this process at one of the node's menu (devA) to trust the other node (devZ) and then goes to the other node (devZ) to establish trust in the first node (devA) as well.

However, establishing mutual trust can become cumbersome if nodes are installed at different times and configured by different responsible people because of restricted access to the node's configuration possibilities. A typical example can be found at the grid connection point of a premises if devices are installed to measure or control electricity: A metering point operator is responsible for a control unit, whereas the user is responsible for an energy management system, or charging station or heat pump etc.

The installation of devices with separate configuration possibilities and the necessity to configure mutual trust can lead to situations where one of the responsible people needs to access one of the devices again on a different day. Apart from the delay, this could create costs if an installer is required for this last step.

To remedy difficulties from such special cases with the classic "SHIP pairing", a "SHIP Pairing Service" methodology is described in this specification. Technically, it is based upon widely used service announcements (using standardised mDNS/DNS-SD) with the service type "shippairing". This version of the specification supports the following scenario:

- devA can receive a shippairing request to trust devZ.
- An administrator configures devZ to trust devA and to send the shippairing request.
- devA will usually accept only one shippairing request (with a valid authentication as described below). If devA later accepts another shippairing request of a new devZ (e.g. because the previous devZ is broken and must be replaced), it will remove trust in the previous devZ (see section 10.3).

In the above-mentioned example, devA would be the user's device (energy management system/charging station/heat pump/...), whereas devZ would be the control unit of the metering point operator.

The shippairing request of devZ is cryptographically authenticated with a secret of devA. This secret had to be conveyed exclusively to devZ beforehand. This way, devA will be able to safely accept a correctly authenticated request of devZ without further user interaction. In addition, certificate fingerprints of devA and devZ are used (instead of SKIs) to establish the mutual trust for SHIP connections.

2 References

2.1 EEBUS documents

[SHIP] *Minimum:* EEBus_SHIP_TS_Specification_v1.0.1.pdf
Recommended: EEBus_SHIP_TS_Specification_v1.1.0.pdf
Elements within this document that rely on the SHIP specification v1.1.0 are marked with "[1.1.0]"

[SRIP] *Minimum:* EEBUS_TS_ShipRequirementsForInstallationProcess_V1.1.0.pdf

2.2 Normative References

The following documents, in whole or in part, are normatively referenced in this document and are indispensable for its application. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

[FIPS 180-4] FIPS PUB 180-4: Secure Hash Standard (SHS), NIST, 2015.

<<http://dx.doi.org/10.6028/NIST.FIPS.180-4>>

[N-SP800-90A] Recommendation for Random Number Generation Using Deterministic Random Bit Generators, NIST Special Publication 800-90A, Revision 1, 2015.

<<http://dx.doi.org/10.6028/NIST.SP.800-90Ar1>>

[RFC 2104] IETF RFC 2104: HMAC: Keyed-Hashing for Message Authentication, 1997

[RFC 2119] IETF RFC 2119: Key words for use in RFCs to indicate requirement levels, 1997

[RFC 6234] IETF RFC 6234: US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF), 2011

[RFC 6762] IETF RFC 6762: Multicast DNS, 2013

[RFC 6763] IETF RFC 6763: DNS-Based Service Discovery, 2013

[SrvNames] IANA: Service Name and Transport Protocol Port Number Registry,

<<https://www.iana.org/assignments/service-names-port-numbers>>

[X.690] ITU-T X.690 Recommendation: Information technology - ASN.1 encoding rules: Specification of Basic Encoding Rules (BER), Canonical Encoding Rules (CER) and Distinguished Encoding Rules (DER), 2021

3 Terms and Definitions

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC 2119].

addCu-request

A shippairing request where the TXT record key "type" is set to "addCu".

DNS-SD

Domain Name System – Service discovery, see [RFC 6763].

IANA

Internet Assigned Numbers Authority.

mDNS

Multicast DNS, see [RFC 6762].

QR code

The term "QR code" is a registered trademark of DENSO WAVE INCORPORATED. "QR code" is the short form for "Quick Response Code" and used for efficient encoding of data into a small graphic.

Among others, the international standard ISO/IEC 18004:2015 specifies the encoding of QR code symbols.

SHIP

Abbreviation of "Smart Home IP". The SHIP specification defines SHIP nodes and the related communication protocol ([SHIP]).

SHIP ID

Globally unique identifier of a SHIP node. This ID is essential for the mDNS/DNS-SD based discovery of a SHIP node.

SHIP Node

A SHIP node is a logical device which communicates via the SHIP protocol.

shippairing request

The content of a created and distributed shippairing service instance.

shippairing service

An mDNS/DNS-SD based service announcement of a "shippairing" service instance.

SKI

The Subject Key Identifier (SKI) is the SHA-1 value of a SHIP node's public key. SKI and public key are parts of a X.509 certificate.

UCS

Universal Character Set.

UTF

UCS Transformation Format. A computing industry standard for the consistent encoding, representation, and handling of text expressed in most of the world's writing systems.

4 Process description

4.1 Roles and responsibilities

Throughout this document, two SHIP nodes devA and devZ are considered for the following "SHIP Pairing Service" process that is also briefly shown in Figure 1:

- devA will receive and evaluate a shippairing request for trusting a declared SHIP device (devZ)*¹.
- devZ is configured by an administrator
 - to trust devA and
 - to send a shippairing request containing information for devA to trust devZ.

*¹: In this version of the specification, each shippairing request is a so-called "addCu-request". An addCu-request is determined by the value "addCu" in the TXT record key "type" (see section 5.4). The following sections describe also cases when devA will not process addCu-requests.

In Figure 1, a QR code of devA contains information that is required to configure devZ for the shippairing process (and SHIP connection) with devA (see chapter 12 for details on the QR code and Annex B for alternatives).

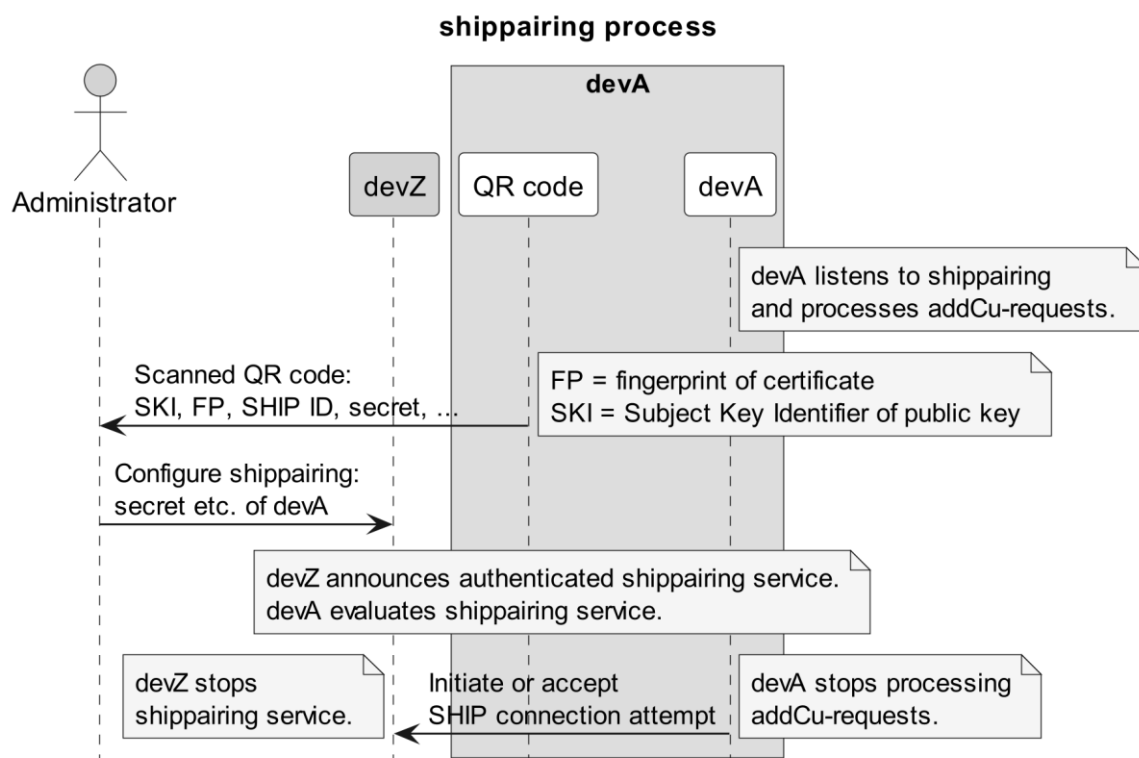


Figure 1: Overview of shippairing process (without SHIP details)

Within this context,

- devA does not announce a shippairing request (to devZ),
- devZ does not receive a shippairing request (of devA).

186 This specification does also not consider the case where a shippairing request is not submitted by
187 devZ itself but by a separate tool or device instead.

188 The shippairing request of devZ requires that certain information on devA is available. In Figure 1,
189 this information is collected by the administrator:

- 190 • The SHIP ID of devA:
191 devZ will need this to find devA to establish a SHIP connection.
- 192 • The fingerprint of devA's certificate:
193 devZ will authenticate devA during the TLS handshake of the SHIP connection (see chapter
194 10).
- 195 • The secret of devA (devA-secret):
196 The shippairing request contains a cryptographical field ("digest") which devA uses to
197 authenticate that the sender knows the correct secret.

198 Note 1: The "SKI" mentioned in Figure 1 belongs to the classic SHIP trust mechanisms, but is not used
199 within the SHIP Pairing Service process. See section 10.1 for more details.

200 Note 2: Without the devA-secret and the fingerprint of devA's certificate, only the classic SHIP trust
201 mechanisms can be used (i.e. SHIP Pairing Service is not possible in this case).

202 Note 3: If the secret of devA is part of the QR code, it is recommended to ensure that the QR code is
203 not easily available. A QR code with devA-secret should only be accessible for the user or installer of
204 the device. For example, a QR code with devA-secret should not be placed on a device's casing.
205 Furthermore, vendors should remind users not to distribute the QR code to other people than the
206 administrator of devZ. The same holds for the devA-secret itself.

207

208 **4.2 Process steps**

209 The following steps apply to the administrator of devZ (see Figure 1):

- 210 1. The administrator collects required information on devA as described in section 4.1.
- 211 2. The administrator configures devZ to trust devA for SHIP communication (SHIP ID of devA,
212 devA-certificate with matching fingerprint).
- 213 3. The administrator configures devZ to create a proper shippairing request.

214 The following steps apply to devZ:

- 215 1. Before devZ sends its shippairing service instance for devA, devZ SHALL trust devA with
216 regards to SHIP connections to and from devA (as configured by the administrator). This
217 includes the responsibility to establish a SHIP connection with devA.
218 Note 1: This is not shown in Figure 1.
219 Note 2: As long as devA does not trust devZ, devZ might need to repeat SHIP connection
220 attempts.
- 221 2. When configured properly, devZ SHALL create a proper shippairing request according to
222 chapter 8.
- 223 3. As soon as devZ successfully completed a SHIP connection with devA (SME state "connection
224 data exchange" reached), devZ SHOULD continue with the announcement of its shippairing

225 service instance for devA until devZ had an uninterrupted SHIP connection with devA for 15
226 minutes^{*1}. Afterwards, devZ SHALL delete its shippairing service instance for devA ^{*2}.

227 ^{*1}: This means that devZ should continue announcing the shippairing service instance for devA if the
228 connection is interrupted before. It also means that the next SHIP connection with devA should start
229 a new timer (beginning with 0 seconds) to measure if that connection lasts for 15 minutes without
230 interruption. The rule applies also if the interruption is caused by a reboot of devZ. On the other
231 hand, devZ can also terminate the announcement of its shippairing service instance for devA before
232 it had an uninterrupted SHIP connection with devA for 15 minutes. However, this reduces the
233 probability for devA of receiving the shippairing service instance in the event that devA has already
234 been configured beforehand to trust devZ's SKI/fingerprint/certificate.

235 ^{*2}: This also means that devZ does not announce this shippairing service instance again after reboot
236 or other kinds of connection interruptions.

237 If the administrator of devZ configures devZ to untrust devA while devZ still announces the
238 shippairing service instance for devA^{*3}, devZ SHALL delete its shippairing service instance for devA
239 immediately.

240 ^{*3} Note: This might occur if the administrator of devZ recognizes a mistake in a first configuration of
241 devZ shortly after the configuration.

242 The following steps apply to devA:

- 243 1. If and only if devA's functionality to receive and evaluate shippairing requests is activated,
244 devA SHALL listen to/search for shippairing requests and evaluate them according to chapter
245 9^{*4,*5}.
- 246 2. If devA was not able to successfully evaluate and accept a shippairing request according to
247 chapter 9, devA SHALL NOT continue with the subsequent steps. Instead, devA SHALL ignore
248 the current announcement of the shippairing service instance. If the service instance is
249 received again, devA begins again with step 1.
250 Note: Sections 5.4 and 5.5 describe rules for the case that devZ wants to correct a shippairing
251 request. The re-evaluation described above helps in case a goodbye message of the service
252 instance has not been detected and a corrected shippairing request uses the same service
253 instance name.
- 254 3. If devA successfully evaluated and accepted an addCu-request, devA SHALL deactivate its
255 processing of addCu-requests (see also section 4.3). Then, dev A SHALL trust the SHIP node
256 with the SHIP ID of the request's key "trustId" together with the related parameters
257 according to chapter 10. devA SHALL then conform to the SHIP specification to establish a
258 SHIP connection with devZ (evaluate incoming SHIP connection attempts of devZ, initiate
259 SHIP connection to devZ).
- 260 4. devA SHALL conform to section 4.3 to deactivate or to reactivate processing addCu-requests.
261 Section 4.2 applies also for a reactivated process, but rules of section 4.3 may also lead to an
262 abortion (deactivation) of the process.

263 ^{*4}: devA conforms to this specification only if the deactivation of the functionality to receive and
264 evaluate shippairing requests is permitted by conditions described in this specification.

*⁵: This specification describes cases when devA deactivates the processing of addCu-requests. In general, there are two possibilities to realise this: The first possibility is to deactivate the functionality to receive and evaluate shippairing requests. The second possibility is to continue receiving and evaluating shippairing requests and abort the evaluation if the shippairing request is recognised as addCu-request. This is described in chapter 9.

Note: This version of the specification only describes addCu-requests. But it can be expected that a successor of this version will contain additional request types.

4.3 addCu-request process deactivation/reactivation at devA

Once devA accepted an addCu-request (of devZ), it is neither necessary nor useful to accept further addCu-requests as long as the SHIP connection (with devZ) is functional. The successful pairing should be preserved and protected from invalid attempts that try to break this pairing. This is even more relevant if devA uses a static secret.

Therefore, devA SHALL stop processing addCu-requests once it accepted an addCu-request and as long as it intends to continue this pairing with devZ. Criteria to potentially discontinue a pairing are considered for this purpose.

The following rules are given to assess if the pairing with the currently trusted devZ (see section 11.4) could (potentially) be discontinued and to begin processing addCu-requests, again. In these rules, the current devZ is denoted as devZ1 and a potential successor is denoted as devZ2:

1. Automatic procedures, mandatory for devA:

- a. An automatic reactivation for processing addCu-requests requires that the following condition is fulfilled:
 - i. devA is not able to perform SHIP Message Exchange with the paired devZ1 since at least 15 minutes (devA is running continuously in this period).
- b. If the condition of item a is fulfilled, exactly one of the following cases can occur:
 - i. If devA receives and successfully evaluates a new valid addCu-request on the new devZ2, devA SHALL untrust the former devZ1, SHALL trust the new devZ2, and SHALL deactivate processing addCu-requests. Then, devZ2 is considered as the current devZ.
 - ii. If otherwise a SHIP connection with devZ1 is suddenly possible again, devA SHALL deactivate processing addCu-requests. This means the trust in devZ1 remains and is not replaced by a trust in a different SHIP node.

2. Manual procedures, optional for devA*¹:

- a. devA has been reset to factory default.
- b. The user of devA deletes the existing pairing to devZ in the user interface.
- c. The user of devA activates the processing of addCu-requests in the user interface, again. In this case, items 1.b.i and 1.b.ii apply as well.

*¹ Note: There might be circumstances where a user of devA needs to remove a pairing between devA and devZ by intention. Vendors should guide users with clear instructions and warnings to prevent that a user accidentally uses such possibilities to remove a pairing with devZ.

4.4 Parallel shippairing requests of devZ

In this section we assume that the network of devA and devZ contains further SHIP nodes devC, devD, etc., which can receive and evaluate shippairing requests like devA. We also assume that the user wants these nodes to trust devZ via SHIP Pairing Service like devA and that devZ gets proper configurations for these devices. Then, it is permissive for devZ to send the required shippairing requests in parallel. However, devZ SHALL announce these requests with individual service instance names (see section 5.2).

devZ SHALL NOT send parallel (different) shippairing requests to a particular devA.

4.5 Handling absence of SHIP Message Exchange at devZ

The behavior of devZ in the event of an extended absence of SHIP Message Exchange with devA is outside the scope of this specification. This specification does not mandate autonomous re-initiation of the shippairing service by devZ and explicitly recommends against it. Any response to such an extended absence of SHIP Message Exchange with devA should be determined by the administrator of devZ.

Rationale: One example for an unsuccessful SHIP connection with devA is a network problem (e.g. broken switch or cable). In this case, fixing the network problem will permit communication, again, without a new shippairing request. Another example is the intentional replacement of devA with another "devA2". devA might still be in the network. In this case, a new shippairing request of devZ to devA could re-establish a pairing that is not intended anymore. Instead, the correct approach would be to not send automatically a new shippairing request to devA and wait for a new configuration by the administrator (in this example with the SHIP ID, secret, fingerprint etc. of devA2).

5 shippairing service

5.1 Introduction

Similar to SHIP, the SHIP Pairing Service concept makes use of mDNS/DNS-SD according to [RFC 6762] and [RFC 6763], resp., to provide a shippairing request of devZ to devA.

Each instance of a shippairing service gets a unique name in the local network. According to [RFC 6763], a service instance name has the form

`<Instance>.<Service>.<Domain>`

The announcement and discovery of service instances includes usage of these record types:

- PTR (pointer record)
Discovery of available services. Maps type of service to service instance name:
`<Service>.<Domain> → <Instance>.<Service>.<Domain>`
- SRV (service record)
Maps service instance name to host name and port (and some more data).
- TXT (text record)
Provides key-value entries for a given service instance names.
- A, AAAA (address records)
Maps host name to IP address.

The following sub-sections will define the shippairing specifics for the names and records.

5.2 Service instance names

For SHIP Pairing Service, the service name "shippairing" is registered at the IANA for the transport protocol "tcp" (see [SrvNames]). Therefore, a shippairing service SHALL use "_shippairing._tcp" (without the surrounding quotation marks) for `<Service>`, and "local." (without the surrounding quotation marks) for `<Domain>`.

devZ can announce its shippairing request with an `<Instance>` part that is completely different from the `<Instance>` part of devZ's SHIP service. However, it is more transparent if the `<Instance>` parts of the SHIP service instance and the SHIP Pairing Service instance(s) have similarities. Therefore, recommendations are given for two cases:

1. If devZ does not announce more than one shippairing request any time:

In this case, the `<Instance>` part for the SHIP Pairing Service SHOULD be the same as the `<Instance>` part for the SHIP service.

2. If devZ announces (or could announce) more than one shippairing request in parallel:

According to section 4.4, it is possible for devZ to announce several independent shippairing requests in parallel. In this case, the recommendation of case 1 cannot be applied directly and needs to be modified: If devZ's SHIP instance label is denoted as `<SHIP-Instance>`, then the shippairing `<Instance>` SHOULD have the form

<SHIP-Instance><Delimiter><Counter>

where <Counter> is a string representing a positive integer value (a counter for the SHIP Pairing Service instance) and <Delimiter> is a character to separate <SHIP-Instance> from <Counter>.

Example: If devZ has a SHIP service with service instance name "Control Unit ExampleCompany C8277H008F._ship._tcp.local." and wants to submit two different shippairing requests (for two different devices) in parallel, then devZ could have two shippairing service instances with the name "Control Unit ExampleCompany C8277H008F#1._shippairing._tcp.local." and "Control Unit ExampleCompany C8277H008F#2._shippairing._tcp.local.", resp.

Note: Creating service instance names with the pattern "<SHIP-Instance><Delimiter><Counter>" is in general advisable as it helps for cases where a TXT record with wrong parameters is to be replaced with corrected values. See sections 5.4 and 5.5 for details.

Note: The structure and content of a service instance name is transparent for devA. I.e. devA does not need to distinguish these cases, nor does it need to verify if a SHIP Pairing Service instance name is similar or identical to a SHIP service instance name.

5.3 General service requirements

According to section 4.4, devZ is responsible to create, announce, and delete its shippairing service instances and to respond to queries related to its shippairing services, whereas devA is responsible to evaluate received and query for shippairing service instances (provided that this is not deactivated according to section 4.3).

DNS-SD records of shippairing service instances SHOULD have a TTL of 2 minutes.

Even though the shippairing service is registered for TCP as transport protocol, this version of the specification does not define a protocol and reserves this possibility for future use. Therefore, devZ SHALL set any unused port number in SRV records for its shippairing service instances. However, devZ SHALL NOT accept any incoming connections on that port and it SHALL NOT submit any data over that port. devA SHALL NOT send data to a port of a shippairing SRV record.

devZ will also set its host name in SRV records. As this version of the specification does not define a communication protocol, devA will not need to resolve this host name to an IP address. However, devZ must provide this possibility nonetheless. It is recommended that devZ uses the same host name as for SHIP records. devA SHALL not use the host name of a shippairing service for automatic relation to a SHIP service of devZ.

5.4 TXT record

devZ SHALL provide for an active shippairing service instance a TXT record according to Table 1.

M/O	Key	Value	Runtime* ¹	Description
M	txtvers	"1"	Static	Version number of the TXT format. Example: txtvers=1
M	parType	"fpSha256"	Static	fpSha256: Indication to use SHA-256 fingerprint of certificate. Example: parType=fpSha256
M	forId	Type: SHIP_ID	Static	devA-SHIP-ID (see section 6.1). Example: forId=i:983327_u:C8277H008F-3
M	forPar	[0-9A-F]{64}	Static	SHA-256 fingerprint of devA-certificate. Example: forPar=C74B7855D3479415F62CC01E5F6D9A93EBC676057D85417ADA16FD1384338943
M	trustId	Type: SHIP_ID	Static	devZ-SHIP-ID (see section 6.1). Example: trustId=i:46925_u:43652bk-2-gt1
M	trustPar	[0-9A-F]{64}	Static	SHA-256 fingerprint of devZ-certificate. Example: trustPar=2CC72E781F7A7D2A08D50196C50FEDF0F7BA583F43F76C8C0DDEC9EEF0D005B4
M	trustCurve	"secp256r1" "brainpoolP256r1" "brainpoolP384r1"	Static	Curve of devZ's certificate public key. Example: trustCurve=secp256r1
M	type	"addCu"	Static	Command* ² . Example: type=addCu
M	trustNonce	[0-9A-F]{32}	Static	devZ-nonce, random "nonce" of devZ. Example: trustNonce=BDCEE427FA7208DF3C1F2A749BA6F4D4
M	alg	"hmacSha256"	Static	algorithm of the digest. Example: alg=hmacSha256
M	digest	[0-9A-F]{64}	Static	Digest on certain keys (see section 7.4) using the algorithm specified in key "alg" and a parameter based upon devA-secret and trustNonce (devZ-nonce). See chapter 7. Example: digest=2CC72E781F7A7D2A08D50196C50FEDF0F7BA583F43F76C8C0DDEC9EEF0D005B4

Table 1: Key-value pairs of a shippairing TXT Record

401 Notes of Table 1:

- 402 • *¹: A shippairing service instance can be considered as the announcement of a one-time
403 event. The runtime refers to the lifetime of the service instance (the "one-time event"). This
404 means that a "static" key value will not change until devZ finally "deletes" the service
405 instance (more precisely, a service is "deleted" by sending a "goodbye" announcement as
406 specified in [RFC 6762]). But it might be that devZ will – at a later time – announce again a
407 shippairing service instance (with potentially the same name) with different values.
- 408 • *²: Informative note on "addCu": The part "Cu" emphasizes that devZ (the SHIP node to
409 "add") is a so-called "Control Unit" or similar device (e.g. a Smart Meter Gateway with
410 control-unit-like functionality) where different people than the devA-user are responsible to
411 configure devZ and establish a SHIP pairing.

412 Rules on columns of Table 1:

- 413 • M/O: "M" denotes a mandatory key. "O" denotes an optional key.
- 414 • Value: All values are described as UTF-8 encoded strings. The following notations define
415 permitted values:
 - 416 o "<a>"
417 <a> is the string value. The surrounding double quotes do not belong to the value.
 - 418 o "<a>" | "" | ...
419 Choice: Any of the listed strings <a> or or ... is a permitted string value.
 - 420 o Type: SHIP_ID
421 The value SHALL conform to the format of a valid SHIP ID.
 - 422 o [0-9A-F]{32}
423 Regular expression for the value: 32 uppercase hexadecimal digits (corresponding to
424 a 128bit value)
 - 425 o [0-9A-F]{64}
426 Regular expression for the value: 64 uppercase hexadecimal digits (corresponding to
427 a 256bit value)
- 428 • Runtime: "Static" means that key and value SHALL NOT be changed by devZ until devZ
429 deletes the shippairing service instance. "Dynamic" means that devZ might update the
430 shippairing service instance with a different value. See also "Rules on shippairing service
431 instance updates" below.

432 Future versions of this specification may add further keys or values.

433 Rules on shippairing service instance updates:

- 434 • devZ SHALL NOT change a "static" key once it created a shippairing service instance. See
435 section 5.5 for possibilities to provide changes of "static" key values and for general
436 possibilities to replace or update a service instance.

437 Rules on specified keys:

- 438 • The key "txtvers" SHALL be the first key in the TXT record.
- 439 • The values of the keys "forId" and "trustId" SHALL be set and interpreted in UTF-8 format.
440 The values SHALL NOT be empty.

- The key "trustCurve" permits values for the elliptic curves secp256r1, brainpoolP256r1 ([SHIP]^{1.1.0}), and brainpoolP384r1 ([SHIP]^{1.1.0}). See section 6.2 to choose a certificate/curve that is supported by devA.
- For each new shippairing request*³, devZ SHALL create a new random number as value for the key "trustNonce" according to section 6.2.
- The key "type" denotes the shippairing request as "addCu-request" if the key is set to "addCu".

*³: The unsolicited announcement of an already existing service instance (according to its TTL or because dynamic keys are updated) is not considered as "new shippairing request".

Rules on the evaluation of a TXT record:

- devA SHALL ignore keys that are not mentioned in Table 1.
- devA SHALL consider a TXT record being invalid if a mandatory key is missing or if a specified key has a value that does not conform to this specification (i.e. the value does not match the rules for permitted values).
- devA SHALL consider a TXT record being invalid if the first key is not "txtvers".
- devA SHALL consider a TXT record being invalid if the value of the key "trustCurve" denotes an elliptic curve that devA does not support for a SHIP connection.

Rules to protect devA-secret:

- devZ SHALL NOT use the devA-secret in any way to create the value of "trustNonce".
- devZ SHALL NOT add the devA-secret or parts of it in any way to any of the other TXT record keys.
- devZ SHALL NOT put any additional key into the TXT record that contains the devA-secret in any way.

Note: The devA-secret is used to create the value of the key "digest". There is currently no known algorithm that can extract the devA-secret from the digest faster than a brute-force approach.

5.5 Correcting an unsuccessful request

By default, a shippairing request expresses a unique task with usually fixed parameters (at least as far as "static" keys of the TXT record are concerned). If a request has accidentally been sent with wrong parameters*¹, this specification follows the approach to retract the first (wrong) "task" and send a new one afterwards. As described below, the preferred option is to assign such corrected shippairing requests a different service instance name than before. Similarly, parallel shippairing requests require as well different service instance names. Therefore, it is in general advisable to append some kind of counter to a service instance name as shown in section 5.2.

*¹ Note: An administrator might configure devZ with a wrong parameter because, e.g., he accidentally mistyped the devA-secret or fingerprint/..., or because he already received a wrong value. In this case, a correction must be possible as described in this section.

478 If devZ sent a shippairing request with "static" keys that are not valid anymore and need to be
479 replaced, devZ SHALL delete the current (invalid) shippairing request first (in mDNS this is called a
480 "goodbye message"). Then, devZ can proceed with one of the following options:

- 481 • Option 1: devZ announces the new (corrected) shippairing request with a different service
482 instance name for the new shippairing request than before. This is the preferred option.
- 483 • Option 2: The old shippairing request had an initial TTL that is denoted as "TTL_old". After
484 the deletion of the old shippairing request, devZ SHALL wait for at least $1.5 * \text{TTL_old}$ before it
485 creates a new shippairing request (with potentially the same service instance name).

486 Rationale: devA might ignore a service instance once it discovered wrong static keys. It might then
487 ignore updates as static keys must not change. The rules above help devA to recognize corrected
488 parameters soon. They also help for cases where mDNS update messages are not received or where
489 a so-called cache-flush bit is not processed correctly.

490 The general possibility to unsolicitedly announce an already existing service instance (according to its
491 TTL or because dynamic keys are updated) remains unchanged.

492

6 Identification and cryptographic requirements

6.1 Identification

SHIP ID:

- According to [SHIP], a SHIP node is identified by its SHIP ID.
- The TXT record key "forId" contains the SHIP ID of devA.
- The TXT record key "trustId" contains the SHIP ID of devZ.

See [SRIP] for the recommended pattern of a SHIP ID.

6.2 Cryptographic parameters

Certificates and curves:

- This specification considers and permits only certificates as specified by [SHIP] for the communication between SHIP nodes, if not stated otherwise.
- Only the certificate's public key algorithms and parameter sets as specified by [SHIP] are permitted in this specification.
 - For [SHIP] version 1.0.x, only the elliptic curve secp256r1 is permitted.
 - For [SHIP] version 1.1.x, elliptic curves secp256r1, brainpoolP256r1, and brainpoolP384r1 are permitted.
- The TXT record key "trustCurve" contains the curve name of devZ's certificate public key.
 - It should be noted that many SHIP implementations might just support the curve secp256r1 (as this is mandatory in the first SHIP specification), but no brainpool curve. Therefore, the administrator of devZ could analyse the QR code of devA to check which elliptic curve is supported by devA (see Annex B for other possibilities).

Fingerprints:

- For the intended SHIP communication between devA and devZ, the respective SHIP certificates of devA and devZ SHALL be selected^{*1}. For each of these certificates, the corresponding fingerprint SHALL be calculated as the SHA-256 value ([FIPS 180-4]) of the certificate in the DER format ([X.690]).

^{*1}: See section 5.4 to select certificates based upon mutually supported curves.

devA-secret:

- devA SHALL have a secret value for SHIP Pairing Service. The value SHALL be an integer value in the range $[0, 2^{128}-1]$.
 - As readable string (e.g. in a QR code), each value of this range is represented by 32 hexadecimal digits.
- Per devA, a random number generator (see section 6.3) SHALL be used to create a practically unique secret for devA.
 - This also means that vendors SHALL do their best effort to prevent that any two different devA instances (SHIP nodes) they produce have the same secret.
- The devA-secret MAY be static through the lifetime of devA.

- It is permissive for the user of devA to change the devA-secret in general. However, this can introduce difficulties in an installation process and should therefore be avoided. The devA-secret SHOULD NOT be changed before an ongoing shippairing process with devZ was completed, as this would require to restart the whole process with the renewed devA-secret for the administrator of devZ. In addition, changing a devA-secret would invalidate a printed QR code containing the secret (see also Note 3 in section 4.1). This should be considered as some day there might be a need for another SHIP Pairing Service process with devZ or a replacement of devZ. See also Annex B.

trustNonce (devZ-nonce):

- As explained in section 5.4, devZ SHALL create a random value for the key "trustNonce". The value SHALL be an integer value in the range $[0, 2^{128}-1]$.
 - Each value of this range is represented by 32 hexadecimal digits in the TXT record (see Table 1).
- Per devA as well as per new SHIP Pairing Service process with a given devA, a random number generator (see section 6.3) SHALL be used to create a practically unique value of "trustNonce".
 - This also means that vendors of devZ (if devZ creates the value on its own) or administrators of devZ (if devZ is configured with a value) SHALL do their best effort to prevent that for any two different devA instances (SHIP nodes) the same value of trustNonce is used.
- devZ SHALL NOT reuse the value of trustNonce of an earlier successful shipPairing request to devA for a later re-execution of a SHIP Pairing Service process with devA.
 - Note: This is in accordance with the rule before which includes the case of a new SHIP Pairing Service process with a given devA.
 - Note: A re-execution can become necessary if devZ or devA have to be reset and then need to be paired again.

6.3 Random number generators

For SHIP Pairing Service, it is most important that neither devA-secret nor trustNonce can be predicted or derived by an adversary from available data. This can be achieved with a proper design of random number generators.

For a given number range $[a, b]$, a random number generator can generate a sequence of randomly chosen values from this range. For random numbers required for SHIP Pairing Service, an ideal random number generator is defined as a source of integer values where all values of the range are uniformly distributed and where it is not possible to predict the next value to be created (i.e. each value of the range has the same probability to appear as next value in the sequence).

Practical random number generators can usually fulfil the requirements on uniform distribution and unpredictability only to some extent. This specification does not prescribe how much a practical random number generator may deviate from the ideal requirements. But it can give rules and recommendations in general and for particular data of the SHIP Pairing Service.

572 General rule:

- 573 • A vendor SHALL do its best effort to use a random number generator that almost fulfils these
574 requirements in a generally accepted manner.

575 Creation of devA-secret:

- 576 • If devA uses its serial number or any other publicly available or derivable static property of
577 devA as input^{*1} of the random number generator to create the devA-secret, then devA
578 SHALL also use another source of sufficient entropy^{*2} to initiate the random number
579 generator.

580 Creation of "trustNonce" value (devZ-nonce):

- 581 • If devZ uses its serial number or any other publicly available or derivable static property of
582 devZ or any publicly available or derivable static property of devA as input^{*1} of the random
583 number generator to create the trustNonce value, then devZ SHALL also use another source
584 of sufficient entropy^{*2} to initiate the random number generator.

585 ^{*1} Note: This is comparable to [N-SP800-90A] where a serial number would be part of a so-called
586 personalization string.

587 ^{*2} Note: Such entropy sources are usually based on physical noise sources or non-deterministic
588 random number generators.

589

7 shippairing digest calculation

7.1 Overview

This chapter describes how the value of the TXT record key "digest" is calculated by devZ as well as devA. It comprises of these steps:

1. Select an algorithm
2. Create a (temporary) key
3. Create a (temporary) message with selected content of the TXT record
4. Calculate the digest

Annex A provides example values for devA, devZ, and the calculation of a digest.

7.2 Select algorithm

The TXT record key "alg" denotes which algorithm is used for the calculation. devZ sets this key to a permitted value, while devA reads this key. In this version of the specification, only the value "hmacSha256" is permitted. This selects the algorithm "HMAC-SHA256", an HMAC according to [RFC 2104] with the hash function SHA-256 according to [FIPS 180-4] ([RFC 6234] contains also code to implement the HMAC and hash function). In general, an HMAC algorithm can be written as

$$\text{HMAC}_K(M)$$

with the key "K" and the message "M".

7.3 Create (temporary) key K

The key K is constructed from devA-secret and devZ-nonce:

1. Applies to devZ only: Create a new random number for devZ-nonce according to section 6.2.
 - a. See section 5.4: When the TXT record is created, devZ puts devZ-nonce properly encoded into the TXT record key "trustNonce".
2. Applies to devA only: devA reads the TXT record key "trustNonce" to get devZ-nonce.
3. Calculate K as direct binary concatenation (operator symbol "||") of the octets of devA-secret and devZ-nonce:
$$K = \text{devA-secret} \parallel \text{devZ-nonce}$$

Example for the concatenation: For two hexadecimal values $a=7A...DE$ and $b=BD...D4$, the concatenation

$$a \parallel b$$

yields the hexadecimal value

$$7A...DEBD...D4$$

Section 5.4 contains rules to protect the devA-secret. As the key K contains the devA-secret, these rules also prohibit devZ to include the key K or parts of it in any way into the TXT record.

7.4 Create (temporary) message M

The (temporary) message M is a UTF-8-encoded string. It is constructed by the ordered string concatenation (symbol "|") of selected TXT records keys (name, assignment, value in a given encoding) and delimiters (semicolon):

```
M = "txtvers=" || <txtvers value: TXT-string> || ";" ||
    "parType=" || <parType value: TXT-string> || ";" ||
    "forId=" || <forId value: TXT-string> || ";" ||
    "forPar=" || <forPar value: TXT-hexadecimal> || ";" ||
    "trustId=" || <trustId value: TXT-string> || ";" ||
    "trustPar=" || <trustPar value: TXT-hexadecimal> || ";" ||
    "trustCurve=" || <trustCurve value: TXT-string> || ";" ||
    "type=" || <type value: TXT-string> || ";" ||
    "trustNonce=" || <trustNonce value: TXT-hexadecimal> || ";" ||
    "alg=" || <alg value: TXT-string> || ";"
```

with the following notation:

- "xyz": The text with the double quotes (xyz in this example, i.e. without the double quotes) is appended to M.
- <t value: encoding>: The value of the TXT record key "t" is appended to M in the same encoding as used in the TXT record.

It is not permissive to deviate from the specified construction of M. This also means that it is not permissive to alter the order of keys (and values) or to add or remove entries.

Example: M is set to the following UTF-8 string (no spaces or line breaks):

```
txtvers=1;parType=fpSha256;forId=i:983327_u:C8277H008F-
3;forPar=C74B7855D3479415F62CC01E5F6D9A93EBC676057D85417ADA16FD1384338943;tr
ustId=i:46925_u:43652bk-2-
gt1;trustPar=2CC72E781F7A7D2A08D50196C50FEDF0F7BA583F43F76C8C0DDEC9EEF0D005B
4;trustCurve=secp256r1;type=addCu;trustNonce=BDCEE427FA7208DF3C1F2A749BA6F4D4;al
g=hmacSha256;
```

7.5 Calculate the digest

For the message M from section 7.4, calculate the HMAC according to section 7.2 with the key K from section 7.3:

$\text{HMAC}_K(M)$

devZ sets the TXT record key "digest" to the hexadecimal representation of the HMAC (see chapter 8). devA compares the value of the TXT record key "digest" (which is a hexadecimal string) with the hexadecimal representation of the HMAC (see chapter 9).

Note: Some tools to calculate an HMAC accept the key K in various input formats. For example, the tool "openssl" provides an option "hexkey:..." to take the hexadecimal encoding of the key K.

See Annex A for an example, with the calculated digest in section A.3.

8 Create shippairing request

This chapter summarises required data and steps for devZ to create a shippairing request to devA.

Creating a shippairing request of devZ to devA means creating a proper shippairing service instance according to chapter 5, including a dedicated TXT record according to section 5.4.

The following information is required as input for devZ:

- SHIP IDs of devA and devZ are required according to section 6.1 to set the keys "forId" and "trustId" properly.
- Fingerprints of the certificates of devA and devZ are authentication parameters. They are required according to section 6.2 to set the keys "forPar" and "trustPar" properly. The key "parType" is set according to section 5.4 to express the kind of authentication parameter*¹.
- devZ sets the key "trustCurve" according to section 6.2. This expresses the curve of devZ's certificate. Section 6.2 also explains to choose only a curve that is also supported by devA.
- devZ must have the devA-secret according to section 6.2.

*¹: This version of the specification only supports SHA-256-based fingerprints of certificates as authentication parameter.

Then, devZ proceeds as follows:

- devZ creates a random number (devZ-nonce) according to section 6.2 to set the key "trustNonce" properly.
- devZ chooses an algorithm for the digest calculation*² and sets this algorithm in the key "alg" according to section 7.2.
- devZ calculates the digest according to chapter 7 and sets the result in the key "digest".

*²: This version of the specification only supports HMAC-SHA256 as algorithm, denoted by the value "hmacSha256" in the key "alg".

devZ sets also further TXT record keys that are not mentioned here (see section 5.4 for required keys and their values).

Finally, devZ creates a shippairing service instance according to chapter 5.

9 Evaluate shippairing request

Evaluating shippairing requests means to listen and search for shippairing service instances according to chapter 5 and to evaluate their TXT records. As long as devA's functionality to receive and evaluate shippairing requests is activated, it monitors for shippairing instances where the key "forId" is set to devA's SHIP ID.

The evaluation of shippairing requests can be aborted by conditions described in sections 4.2 and 4.3 at any time.

devA SHALL NOT evaluate two or more shippairing requests in parallel. The following steps apply per shippairing instance.

1. Analyse relevance for devA:

devA SHALL continue with the evaluation of a TXT record if and only if all of the following conditions are fulfilled:

- The key "txtvers" is the first key in the TXT record and has the value as specified in section 5.4.
- The key "forId" is present and set to devA's SHIP ID.
- The key "parType" is present and set to a value supported by devA.
- The key "forPar" is present and – together with the key "parType" – uniquely identifies a SHIP certificate of devA.
- The value of the key "trustCurve" is present and set to a value supported by devA.
- If the TXT record key "type" is set to "addCu": devA's functionality to process addCu-requests is activated*¹.
- The TXT record fulfils requirements of section 5.4 in general (i.e. all mandatory keys are present and contain permitted values).

*¹ Note: This means the evaluation of the shippairing request is aborted if devA currently does not process addCu-requests. See sections 4.2 and 4.3 for more details.

2. Only process new requests:

Chapter 11 describes a ring buffer of devA that is used to store the last successful shippairing requests. Only new shippairing requests can be added, while known shippairing requests will be ignored. This is achieved as follows:

Each entry of the ring buffer is either unset or contains a pair of algorithm identifier and digest, i.e. { alg, digest }. From the received shippairing request's TXT record, devA creates a (temporary) pair

new-pair = { <value of key "alg">, <value of key "digest"> }

If the ring buffer already contains an entry with this pair, then devA SHALL abort the evaluation of this shippairing request. Otherwise, devA continues with the evaluation of the TXT record.

The new pair SHALL NOT be added to the ring buffer at this stage.

- 727 3. Verify validity of digest:
- 728 • devA reads the value of the key "trustNonce" to get devZ-nonce.
- 729 • devA reads the value of the key "alg" to get the algorithm for the digest calculation*¹.
- 730 • devA calculates the digest according to chapter 7.
- 731 • If and only if the calculated digest is identical to the (converted) value of the TXT record key
- 732 "digest", devA continues with the next step. Otherwise, devA SHALL abort the evaluation of
- 733 this TXT record.

734 *¹: This version of the specification only supports HMAC-SHA256 as algorithm, denoted by the value

735 "hmacSha256" in the key "alg".

736

- 737 4. Store the new digest persistently:
- 738 • devA SHALL add "new-pair" to the ring buffer as described in section 11.3.

739

740 At this point, devA successfully evaluated and accepted a shippairing request.

741 Note: This criterion is used in section 4.2.

742

10 Update SHIP trust

10.1 Introduction

The SHIP Pairing Service concept has been designed as additional trust mechanism that can be added to an existing SHIP 1.0 or SHIP 1.1 implementation. As existing SHIP 1.0/1.1 implementations focus on SKI verification during well-known SHIP trust mechanisms, SHIP Pairing Service requires an adjustment of the SHIP TLS handshake.

Note: Even though the usage of SKIs in SHIP still provides a very high degree of security (no known weakness at the time of writing, see section C.1), regulation authorities on information security are pushing algorithms with even higher cryptographical security to compensate for advances in the development of systems and methodologies that try to break cryptographically protected data. Therefore, SHIP Pairing Service supports the migration towards SHA-256 fingerprints of certificates (see section C.2 on SHA-256 fingerprints and section C.3 on SHIP Pairing Service specific aspects).

10.2 Verification of fingerprint during TLS handshake

During the TLS handshake of a SHIP connection, both SHIP nodes present their respective certificate. For conventional SHIP implementations, the SKI can be extracted from the certificate and can be calculated from the public key (and they must be identical).

For SHIP Pairing Service, the fingerprint of the certificate received during the TLS handshake will be calculated instead (as explained in section 6.2) and verified with the expected fingerprint of the shippairing request. If the fingerprints match, this SHALL be seen like a successful trust in an SKI from the perspective of the SHIP specification.

Note: An implementation might also extract and store the SKI from the certificate upon successful verification of the fingerprint. However, this is not required and not considered as part of the authentication of the other SHIP node.

10.3 Remove SHIP trust of past devZ

As explained in chapter 1, accepting a new shippairing request includes removal of trust in previous shippairing requests. In general, devA SHALL only trust the current devZ with regards to SHIP Pairing Service trust establishment. The current devZ is defined according to section 11.4.

Note 1: This means that devA will never consider more than one SHIP node as devZ.

Note 2: The restriction to accept only one current devZ is closely related to the TXT record key "type" value "addCu" (addCu-request). Subsequent versions of this specification might extend the concept for further installation scenarios.

10.4 Single trust store for SHIP and SHIP Pairing Service (informative)

This section is informative, i.e. it does not define further rules. It rather clarifies several responsibilities of SHIP and SHIP Pairing Service concerning the handling of trust information.

780 A SHIP implementation has a storage to keep its trust and related data to other SHIP nodes
781 persistently. This storage is called trust store.

782 SHIP Pairing Service does neither define nor require a separate trust store. Even the ring buffer of
783 devA (see chapter 11) is not a trust store. It just serves to prevent replay attacks.

784 This means a SHIP implementation is responsible to store trust from a shippairing request like it
785 stores trust from classic SHIP trust mechanisms.

786 The user of devA might trust devZ with a classic SHIP trust mechanism (e.g. discovery via mDNS/DNS-
787 SD und user verification of devZ), i.e. before a shippairing request of devZ is received. If afterwards a
788 shippairing request of devZ is received and successfully evaluated, devA can find a matching SHIP ID
789 in the trust store and can add potentially new information from the shippairing request^{*1,*2}, such as
790 the fingerprint of devZ and the information that devZ has been added with a shippairing request with
791 type "addCu" (control unit or similar device, see section 5.4)^{*3}. Furthermore, devA adjusts the ring
792 buffer.

793 In the given example, it might even happen that devA and devZ successfully communicate via SHIP
794 with each other before the shippairing request is received. But as specified in section 4.2, devA will
795 deactivate its process of receiving and evaluating shippairing requests only upon successful
796 evaluation and acceptance of a shippairing request.

797 If the user of devA removes the trust to devZ, this will reactivate the SHIP Pairing Service process,
798 again.

799 ^{*1} Note: The SHIP ID is not protected by cryptographical algorithms. But stored cryptographical
800 parameters (SKI, fingerprint, etc.) need to be verified on every SHIP connection attempt and lead to
801 an abortion of the communication if the verification fails.

802 ^{*2}: The SHIP ID serves as unique identification throughout the lifetime of a SHIP node (hence, is
803 considered as primary identifier in the trust store).

804 ^{*3}: This is especially important as a removal of this specific device from the trust store will trigger
805 devA to reactivate its processing of addCu-requests.

806

11 Ring buffer of devA

11.1 Introduction

A ring buffer (also known as circular buffer) is used to prevent so-called replay attacks where an adversary could try to interfere a SHIP Pairing Service process by re-using an old (recorded) TXT record. It can store at least the last ten successful shippairing requests (esp. their digests). Only new shippairing requests will be added while replay attacks would be detected.

This chapter defines the ring buffer as logical functionality. Vendors MAY implement this functionality with equivalent concepts.

Note: Storing only ten entries can lead to the situation that at some day the oldest entry in the ring buffer is overwritten. But this is expected to be a very unlikely case as this would usually require that a devZ was replaced at least eleven times for the same devA. It is even more unlikely that an adversary was present already at the first attempt and recorded it just to replay it after the tenth replacement of devZ. Furthermore, the adversary would need to be faster than the new devZ with its legit shippairing request.

11.2 Definition of ring buffer and "next"

devA SHALL have a ring buffer "digest-ring-buffer" with at least 10 entries. The entries are contiguously indexed from 1 upwards (1, 2, ..., last index ≥ 10).

Each entry is either unset or contains the pair (structure)

{ alg, digest }

where "alg" denotes the algorithm of "digest" and "digest" contains a received and verified digest of a shippairing request.

For the ring buffer an index variable "next" is defined. "next" is set to the index (entry) of the ring buffer where a new pair { alg, digest } would be stored.

devA SHALL store "digest-ring-buffer" and "next" persistently.

Note: The ring buffer does not store the secret that was used to verify a digest.

11.3 Adding a new entry to the ring buffer

If a new pair of "alg" and "digest" is to be added to the ring buffer:

- devA SHALL set the entry "next" of "digest-ring-buffer" to the new pair { alg, digest }:
 - digest-ring-buffer[next] := { alg, digest }
- devA SHALL increase "next" by 1. If the result is greater than the last index of "digest-ring-buffer", devA SHALL set "next" to 1.
- devA SHALL store "digest-ring-buffer" and "next" persistently.

11.4 Get entry of currently trusted devZ

As already mentioned in chapter 1, devA will trust at most one devZ from past shippairing requests.

This is either the most recently trusted devZ (the "currently trusted devZ") or there is no trusted devZ yet. The corresponding ring buffer entry can be obtained as follows:

- The index variable "current" is defined as the index variable "next" minus 1:
 - $current = next - 1$
- If the result is smaller than 1, "current" is set to the last index of the ring buffer.
- If the entry at the index "current" is unset, devA has currently no trusted devZ from a shippairing request. Otherwise, the entry at the index "current" belongs to the devZ that is currently trusted.

12 SHIP QR code extensions

The basic SHIP QR code is defined in [SHIP] and has been extended in [SRIP].

For certificate fingerprints used in SHIP Pairing Service, the extensions shown in Table 2 are defined.

Similar to keywords for SKIs, the keyword for fingerprints depends on the supported curve.

Curve	Keyword for SKI	New: Keyword for fingerprint
secp256r1	SKI	FPH256
brainpoolP256r1* ¹	BSKI* ¹	BFPH256
brainpoolP384r1* ¹	B2SKI* ¹	B2FPH256

Table 2: SHIP QR code extensions for fingerprints

*¹: Defined since SHIP version 1.1.0.

For devA implementations supporting this specification, at least one of the following strings SHALL be part of the SHIP QR code:

- FPH256:<SHA-256 fingerprint of certificate (secp256r1)>;
- BFPH256:<SHA-256 fingerprint of certificate (brainpoolP256r1)>;
- B2FPH256:<SHA-256 fingerprint of certificate (brainpoolP384r1)>;

where

<SHA-256 fingerprint of certificate (...)>

is the uppercase hexadecimal representation of the corresponding fingerprint (64 uppercase hexadecimal digits).

Rules of [SHIP] and [SRIP] on mandatory parts of a QR code remain unchanged. This also means that the presence of a fingerprint in the QR code does not overrule requirements on the presence of an SKI.

If a QR code contains the SKI of a given curve, the QR code SHALL also contain the related fingerprint.

Note: If SHIP mandates to support, e.g., secp256r1 in the implementation and "SKI" (secp256r1) part in the QR code, this requires also put the "FPH256" fingerprint into the QR code.

Furthermore, the devA-secret SHALL be part of the QR code (see also Note 3 in section 4.1):

- SPSEC:<devA-secret>;

where

<devA-secret>

is the uppercase hexadecimal representation of the corresponding devA-secret (32 uppercase hexadecimal digits).

An example QR code can be found in section A.1.

Annex A – Example values

A.1 Example values for devA

SHIP ID of devA:

i:983327_u:C8277H008F-3

Secret of devA (devA-secret):

7A37DCF81BDB50F8E92CFA4160CCB3DE

Certificate of devA in printable PEM encoding:

```
-----BEGIN CERTIFICATE-----
MIICJjCCAcugAwIBAgIURCTotMcqCC1Dt8VqkoywEXiXX2QwCgYIKoZIzj0EAwIw
aDELMaKGA1UEBhMCREUxDDAKBgNVBAgMA05SVzEQMA4GA1UEBwwHQ29sb2duZTEV
MBMGA1UECgwMRVhBTvBMRUJSQU5EMSIwIAYDVQQDDb1FRUIwMMWRldkE4MTQtQzgy
NzdIMDA4Ri0zMBA4XDTI1MDgxMTE2NTQwOV0xDTI1MDgxMTE2NTQwOV0wADELMaK
GA1UEBhMCREUxDDAKBgNVBAgMA05SVzEQMA4GA1UEBwwHQ29sb2duZTEVMBMGA1UE
CgwMRVhBTvBMRUJSQU5EMSIwIAYDVQQDDb1FRUIwMMWRldkE4MTQtQzgyNzdIMDA4
Ri0zMfKwEwYHkoZiZj0CAQYIKoZIzj0DAQcDQgAEsKOrknwvjOfO+ihEu9EIIwhM
rJraXone1kWiRA1DMVZK7A1k0rlEmCVL1XBOM7/wcIoIrXnYX7iCqN3oBuarSKNT
MFEWwHQYDVR0OBBYEfKGFVqJD9Xyhew1Xelm3EbeetIVsMB8GA1UdIwQYMBaAFKgF
VqJD9Xyhew1Xelm3EbeetIVsMA8GA1UdEwEB/wQFMAMBAf8wCgYIKoZIzj0EAwID
SQAwwRgIhAic7Is5jXiphSqXqK0V4yu5H2D+JxHBuLWDb74TnsKXOAiEAI0yLNR4z
YyfuxNevdBQr+PkpeskDrpJaAaUPg81JlQg=
-----END CERTIFICATE-----
```

There are several tools (e.g. openssl) that can convert a PEM file to a DER file. Please consider that the fingerprint calculation must be done on the DER encoding, not the PEM encoding!

SHA-256 fingerprint of (DER encoded) devA-certificate:

C74B7855D3479415F62CC01E5F6D9A93EBC676057D85417ADA16FD1384338943

Supported curve of devA:

secp256r1

QR code string of devA:

```
SHIP;SKI:A805 56A2 43F5 7CA1 7B0D 577A 59B7 11B7 9EB4
856C;ID:i:983327_u:C8277H008F-
3;BRAND:EXAMPLEBRAND;TYPE:EMS;MODEL:EEB01devA814;SERIAL:C8277H008F-
3;CAT:2;FPH256:C74B7855D3479415F62CC01E5F6D9A93EBC676057D85417ADA16FD138
4338943;SPSEC:7A37DCF81BDB50F8E92CFA4160CCB3DE;ENDSHIP;
```

915 QR code of devA:



916

917 *Figure 2: QR code example for devA*

918

919 A.2 Example values for devZ

920 SHIP ID of devZ:

921 `i:46925_u:43652bk-2-gt1`

922 Random value of devZ (devZ-nonce):

923 `BDCEE427FA7208DF3C1F2A749BA6F4D4`

924 Certificate of devZ in printable PEM encoding:

```

925 -----BEGIN CERTIFICATE-----
926 MIICLjCCAdOgAwIBAgIUN8jFQuYTDfjInHKhQQQkNVMkHRAwCgYIKoZIzj0EAwIw
927 bDELMAkGA1UEBhMCREUxDDAKBgNVBAGMA05SVzEQMA4GA1UEBwwHQ29sb2duZTEa
928 MBGGA1UECgwRT1RIRVJFWEFNUExFQlJBTKQxITAfBgNVBAMMGEVVFQjAyc3RiNTQt
929 NDM2NTJiaY0yLWd0MTAeFw0yNTA4MTExNTQ4MjlaFw0yOTA4MTExNTQ4MjlaMGwx
930 CzAJBgNVBAYTAkRFRmQwwCgYDVQQIDANOUlcxEDAOBgNVBACMB0NvbG9nbmUxGjAY
931 BgNVBAoMEU9USEVSRVhBTVMRUSJSQU5EMSEwHwYDVQQDDbhFRUIwMnN0YjU0LTQz
932 NjUyYmstMlndDEWWTATBgqhkhjOPQIBBggqhkhjOPQMBBwNCAARB4qKF7gkB0dpc
933 p7aGTT3YnAX2Im5HE3mtqKzDOuPoS1+YG19hpumsol1J4T6iwjX2oORjrxFPmR3P
934 Pd1TAOX7o1MwUTAdBgNVHQ4EFgQUvehlF5Rzk0Fr3BVbMzgiQyN3FRcwHwYDVR0j
935 BBGwFoAUvehlF5Rzk0Fr3BVbMzgiQyN3FRcwDwYDVR0TAQH/BAUwAwEB/zAKBgq
936 hkjOPQQDAgNJADBGAIeAmFBWkuUyLe9546+5uwpYuOVi7fk6ZI3ahNhAs+FQJJMC
937 IQCqkzJAXrjNHJ3dcyQGsP5CSskTrVtTbqnTR3DbxvxAQQ==
938 -----END CERTIFICATE-----

```

939 There are several tools (e.g. openssl) that can convert a PEM file to a DER file. Please consider that
 940 the fingerprint calculation must be done on the DER encoding, not the PEM encoding!

941 SHA-256 fingerprint of (DER encoded) devZ-certificate:

942 `2CC72E781F7A7D2A08D50196C50FEDF0F7BA583F43F76C8C0DDEC9EEF0D005B4`

943 Selected curve of devZ:

944 `secp256r1`

945

946 A.3 shippairing request preparations of devZ for devA

947 Algorithm for digest calculation (see section 7.2):

948 HMAC-SHA256

949 According value for TXT record key "alg":

950 hmacSha256

951 Temporary key K (see section 7.3):

952 7A37DCF81BDB50F8E92CFA4160CCB3DEBDCEE427FA7208DF3C1F2A749BA6F4D4

953 Temporary message M (see section 7.4):

954 txtvers=1;parType=fpSha256;forId=i:983327_u:C8277H008F-
955 3;forPar=C74B7855D3479415F62CC01E5F6D9A93EBC676057D85417ADA16FD138433894
956 3;trustId=i:46925_u:43652bk-2-
957 gt1;trustPar=2CC72E781F7A7D2A08D50196C50FEDF0F7BA583F43F76C8C0DDEC9EEF0D
958 005B4;trustCurve=secp256r1;type=addCu;trustNonce=BDCEE427FA7208DF3C1F2A7
959 49BA6F4D4;alg=hmacSha256;

960 Calculated digest:

961 BCBB62B2176DA2CEE545784CEB1F2A55E049451B12A549C98E8CA213F001DA25

962

Annex B – Pairing without QR code (informative)

QR codes had been introduced to support initial pairings between SHIP nodes. However, a printed QR code becomes outdated when a SHIP node must update its certificate. For SHIP Pairing Service, this is also true if a devA-secret is changed. Therefore, [SRIP] strongly recommends to provide mechanisms to update a QR code if needed. A device's display or a smartphone app are mentioned as example.

More precisely, it is recommended to provide mechanisms to get current parameters for the SHIP pairing, such as SKIs and certificate fingerprints, that can then be used directly for the pairing process or to create an updated QR code.

Rationale: The mechanisms of SHIP also provide possibilities to obtain the majority of parameters with a vendor independent tool by analysing the mDNS/DNS-SD information at the LAN and by so-called TLS probing with a SHIP node to get the node's certificates (and this way the SKIs and fingerprints). However, a devA secret is not available this way. This means:

- As long as a devA-secret remains unchanged, a (hypothetical) vendor independent tool could be used to gather current parameters for the pairing process.
- If devA permits its user to change the devA-secret, a vendor specific tool would be required to gather current parameters for the pairing process – including the current devA-secret.

Annex C – Security aspects

C.1 Security of SHIP SKIs (informative)

This section just gives some background information on SKIs. It does not contain any protocol or implementation relevant information.

Within SHIP, an SKI is the SHA-1 value of a public key. The SHA-1 algorithm has undergone deep mathematical investigations with an increasing number of possibilities to reduce the security level for certain applications – but this does not apply to its usage within SHIP.

Several attacks on SHA-1 reported in the past years primarily focus on so-called collision-attacks. Such attacks are relevant if there is a need to trust an X.509 certificate by verification of its digital signature. Due to discovered weaknesses of SHA-1 with regards to collision attacks, certificates cannot be trusted anymore if their digital signature is based upon SHA-1.

However, SHIP does not rely on the verification of digital signatures. There is also no other part in SHIP where collision attacks could play a role. Instead of collision attacks, an adversary would need to break the so-called second-preimage resistance of SHA-1 to derive a new public key with identical SKI as the one of another device. Up to the time of writing, SHA-1 still provides a second-preimage resistance of 160 bits (i.e. 2^{160} operations required to break it) for short messages (if the public key is seen as message).

The effort to attack a given SKI might be even higher as it is not sufficient to just derive a secondary public key that has the same SHA-1 value. The reason is that a functional TLS communication is only possible if the adversary has also a proper private key for this secondary public key – but the only known possibility to derive a private key from a public key is a brute force attack. The curve secp256r1 has almost 2^{256} points, which means that approximately $2^{256}/2^{160}=2^{96}$ points should have a wanted SHA-1 value – but the remaining $2^{256}-2^{96}$ points (which is still much closer to 2^{256} than to 2^{255}) are those points that do not yield the wanted SHA-1 value. This means an adversary should expect that much more than 2^{160} operations could be required to find a matching private-public key pair for the wanted SKI.

This means there is currently no realistic attack possibility with regards to the use of SHA-1 based SKIs. Nevertheless, the general decrease of acceptance of SHA-1 leads to the need to consider alternatives like certificate fingerprints, see section C.2.

C.2 Certificate fingerprint

Despite the high security level of the application of SKIs within SHIP (see section C.1), there is general tendency to migrate from SHA-1 to improved algorithms. For SHIP Pairing Service, this will include usage of SHA-256 based certificate fingerprints.

A SHA-1 based SKI can be seen as identification of a public key. A SHA-256 fingerprint of a certificate identifies a certificate. As a public key is part of a certificate and as public keys must be unique per certificate, a certificate's fingerprint identifies a public key likewise.

The SHA-256 fingerprint of a certificate is the SHA-256 value of the DER-encoded certificate.

1019 A certificate fingerprint only covers the certificate itself and not any potential further certificate of a
1020 certificate chain. This means even if a certificate "X" belongs to a PKI and is transmitted together with
1021 the chain of intermediate certificates and the CA certificate, the fingerprint belongs only to the
1022 certificate "X" itself.

1023 Notes:

- 1024 • Unlike an SKI, a certificate fingerprint is not part of the certificate itself.
- 1025 • For a certificate of 2kB size (DER encoded), SHA-256 provides a second-preimage resistance
1026 of 251 bits.

1027

1028 **C.3 shippairing as migration step from SKI to fingerprint**

1029 Simply put, the SHIP versions 1.0/1.1 are based upon trust in and verification of SKIs. As regulation
1030 authorities will ban the use of SKIs for authentication purposes in certain environments or
1031 applications, this version of the SHIP Pairing Service specification will only use SHA-256 fingerprints
1032 instead. Nevertheless, SKIs still play an important role and are therefore still required in SHIP QR
1033 codes.

1034